

Launch One ec2 instance and connect with mobaXterm → sethostname Terraform

Got to Github devopsScript by reayaz → [terrafomAL2.sh](https://github.com/reayaz/terraformAL2.sh)

Copy the code and create the file in the ec2-instance Paste it→ name as [terr.sh](#)

Run → sh [terr.sh](#)

```
rm -rf *
```

```
clear
```

```
Ls -al
```

```
```sh
terraform init
```
```

```
```sh
cat <<EOF > main.tf
provider "aws" {
  region = "ap-south-1"
}
EOF
```
```

```
```sh
terraform init
```
```

```
```sh
cat <<EOF > main.tf
variable "fruits" {
  type    = list(string)
  default = ["apple", "banana", "mango"]
}

```

```
output "list_length" {
  value = length(var.fruits)
}
EOF
```
```

```
```sh
terraform validate
```
```

```
```sh
terraform apply --auto-approve

```

```
...
```

```
```sh
cat <<EOF > main.tf
variable "fruits" {
  type    = list(string)
  default = ["apple", "grapes"]
}

variable "vegies" {
  type    = list(string)
  default = ["onions", "tomatoes"]
}

output "combined_list" {
  value = concat(var.fruits, var.vegies)
}
EOF
```
```

```
terraform apply --auto-approve
```

```
cat <<EOF > main.tf
variable "fruits" {
  type    = list(string)
  default = ["apple", "cherry", "kiwi"]
}

output "selected_element" {
  value = element(var.fruits, 1)
}
EOF
```sh
cat <<EOF > main.tf
variable "keys" {
  type    = list(string)
  default = ["name", "age"]
}

variable "values" {
  type    = list(any)
  default = ["puneeth", 22]
}

output "my_map" {
  value = zipmap(var.keys, var.values)
}
EOF
```

```

...
```sh
cat <<EOF > main.tf
variable "keys" {
  type    = list(string)
  default = ["name", "age"]
}

variable "values" {
  type    = list(any)
  default = ["puneeth", 22]
}

output "my_map" {
  value = zipmap(var.keys, var.values)
}
EOF
...
```sh
cat <<EOF > main.tf
variable "my_map" {
  type    = map(string)
  default = {
    "name" = "venky"
    "age"  = "50"
  }
}

output "value" {
  value = lookup(var.my_map, "name")
}
EOF
...
```sh
cat <<EOF > main.tf
variable "fruits" {
  type    = list(string)
  default = ["apple", "mangoes", "orange"]
}

output "joined_string" {
  value = join(",", var.fruits)
}
EOF
...
cat <<EOF > main.tf
variable "unique_names" {
  type    = set(string)

```

```

    default = ["Alice", "Bob", "Charlie"]
}

output "unique_names_list" {
    value = tolist(var.unique_names)
}
EOF
```sh
cat <<EOF > main.tf
variable "fruits" {
    type    = list(string)
    default = ["apple", "mangoes", "orange"]
}

output "joined_string" {
    value = join(",", var.fruits)
}
EOF
```sh
cat <<EOF > main.tf
variable "instance_tags" {
    type = map(string)
    default = {
        Name = "WebServer"
        Env  = "Production"
    }
}

output "instance_tags_output" {
    value = var.instance_tags
}

output "instance_name" {
    value = var.instance_tags["Name"]
}

output "tags_keys" {
    value = keys(var.instance_tags)
}

output "tags_values" {
    value = values(var.instance_tags)
}
EOF
```

```

Vi [main.tf](#)

...

```hcl

```
provider "aws" {  
  region = "ap-south-1"  
}
```

```
variable "instance_count" {  
  description = "number of ec2 instances"  
  type        = number  
  default     = 3  
}
```

```
variable "instance_ami" {  
  description = "AMI to be used"  
  type        = string  
  default     = "ami-08fe5144e4659a3b3"  
}
```

```
variable "instance_type" {  
  description = "Instance type to be used"  
  type        = string  
  default     = "t2.micro"  
}
```

```
variable "instance_name" {  
  description = "Name of the instance"  
  type        = string  
  default     = "TF-Server"  
}
```

```
resource "aws_instance" "myinstance" {  
  count      = var.instance_count  
  ami       = var.instance_ami  
  instance_type = var.instance_type  
  
  tags = {  
    Name = var.instance_name  
  }  
}
```

...

...

Vi [variables.tf](#)

```
```hcl
variable "instance_count" {
  description = "number of ec2 instances"
  type       = number
  default    = 3
}
```

```
variable "instance_ami" {
  description = "AMI to be used"
  type       = string
  default    = "ami-08fe5144e4659a3b3"
}
```

```
variable "instance_type" {
  description = "Instance type to be used"
  type       = string
  default    = "t2.micro"
}
```

```
variable "instance_name" {
  description = "Name of the instance"
  type       = string
  default    = "TF-Server"
}
...

```

Delete all variables from [main.tf](#) file

Only keep provider resources section in [main.tf](#) file

Then terraform validate , terraform plan , terraform apply --auto-approve

Remove all the values of variable from [variable.tf](#) Then create vi terraform.tfvars file add all value in new file called terraform.tfvars

Add all variables like →

```
```hcl
instance_count = 1
instance_ami   = "ami-08fe5144e4659a3b3"
instance_type  = "t2.micro"
instance_name  = "TF-Server"
...

```